Pharmavigil: Drug Adverse Reaction & Medicine Safety Tracking System

Comprehensive Engineering Specification & Academic Project Guide

- **Course:** Database Concepts for Data Science Lab (DCDS Lab)
- **Department:** Artificial Intelligence & Machine Learning (AIML)
- **Evaluation:** End Semester Examination (ESE) Project (10 Marks / 40% Weightage)
- **Team Size:** 4 Members (Same Division)
- **Primary Tech Stack:** Python 3.10+, PostgreSQL / MySQL (SQL), MongoDB (NoSQL), Streamlit, Pydantic, Pytest

1. Executive Summary & Registration Details

1.1 Project Metadata (For Registration Form)

- **Application / Project Title:** `Pharmavigil: Drug Adverse Reaction and Medicine Safety Tracking System`
- **Application Domain:** Healthcare, Pharmacovigilance & Safety Data Science
- **Problem Statement:**

> When pharmaceutical products enter the market, rare adverse drug reactions (ADRs) and manufacturing batch defects only manifest across large patient cohorts. Regulatory bodies require strict transactional tracking of licensed manufacturers, drug formulations, batch numbers, and hospital stock distribution (ACID relational data). Simultaneously, patient-reported adverse reactions and clinical symptoms are irregular, multi-symptomatic, and polymorphic (unstructured document data). **Pharmavigil** bridges this gap: it combines an ACID-compliant relational core for pharmaceutical supply chains with a high-throughput NoSQL document store for adverse event telemetry. The platform computes real-time signal detection, severity aggregations, and automates emergency batch recall procedures across healthcare networks.

- **Sustainable Development Goals (SDGs):**

1. **SDG 3: Good Health and Well-Being** (*Target 3.b: Support research, safety, and supply of essential medicines*).
2. **SDG 9: Industry, Innovation, and Infrastructure** (*Target 9.5: Upgrade technological capabilities in pharmaceutical safety and monitoring*).

2. Evaluation Rubric & Scoring Strategy (10 / 10 Marks)

Criterion	Marks	Deliverable in Pharmavigil	Key Technical Evidence
1. Application Design & Functionality	**2.0**	Interactive 3-Portal Streamlit Web Application	Doctor/Patient Reporting Portal, Hospital Pharmacist Inventory Portal, Safety Officer Admin Control Center
2. Python Programming Implementation	**2.0**	Object-Oriented Design, File I/O & Custom Exceptions	Classes (`Medicine` , `Batch` , `AdverseReport` , `SafetyAuditor`), CSV bulk batch intake, PDF/CSV report generation, domain exceptions
3. SQL & NoSQL Database Integration	**2.0**	Hybrid Dual-Database Architecture	PostgreSQL/MySQL relational schema (5 normalized tables) + MongoDB `adverse_reports` polymorphic collection via `pymongo`
4. Advanced Database Querying	**2.0**	Complex SQL Operations & MongoDB Aggregations	Multi-table JOINs, CTE distribution tracing, SQL Stored Procedure (`sp_recall_batch`), 5-stage MongoDB Aggregation Pipeline, Text Indexing
5. Code Quality, UI & Documentation	**1.0**	PEP 8 Modular Architecture & Streamlit UI	Clean separation into `database/` , `models/` , `services/` , and `ui/` , type hints, docstrings, comprehensive README
6. Testing, Validation & Security	**1.0**	Input Validation, Defensive Queries & Pytest Suite	Pydantic schema validation, parameterized SQL queries, password hashing (`bcrypt`), automated unit tests in `tests/`

3. Dual-Database Architectural Design

3.1 The Architectural Rationale (Viva Goldmine)

A common viva question is: "*Why not store everything in a single SQL or NoSQL database?*"

- **Relational SQL Database:** Pharmaceutical regulation mandates strict ACID compliance. A batch must link to a valid manufacturer, expiration dates must be enforced, and distribution counts cannot become negative. Foreign keys prevent orphaned records.
- **NoSQL MongoDB Database:** Adverse drug reports vary widely between patients:
    - Patient A: Age 28, 1 symptom (`Nausea`), no notes.
    - Patient B: Age 64, 4 symptoms (`Arrhythmia`, `Hypotension`, `Dizziness`, `Dyspnea`), lab findings, free-text doctor observations.
    - Altering an SQL schema every time a new clinical observation metric is introduced violates normalization; MongoDB document collections accommodate polymorphic JSON natively.

4. Relational Database Schema (SQL: PostgreSQL / MySQL)

4.1 Table Definitions & Relationships

1. `manufacturers`:
    - `manufacturer_id` (INT, PK, AUTO_INCREMENT)
    - `company_name` (VARCHAR(150), NOT NULL)
    - `license_number` (VARCHAR(50), UNIQUE, NOT NULL)
    - `country` (VARCHAR(60))
    - `contact_email` (VARCHAR(100))
2. `medicines`:
    - `medicine_id` (INT, PK, AUTO_INCREMENT)
    - `commercial_name` (VARCHAR(150), NOT NULL)
    - `generic_name` (VARCHAR(150), NOT NULL)
    - `dosage_form` (VARCHAR(50) - Tablet, Syrup, Injection, Capsule)
    - `strength` (VARCHAR(50) - e.g., '650mg', '10ml')
    - `manufacturer_id` (INT, FK -> `manufacturers.manufacturer_id`)
3. `batches`:
    - `batch_number` (VARCHAR(50), PK)
    - `medicine_id` (INT, FK -> `medicines.medicine_id`)
    - `manufacturing_date` (DATE, NOT NULL)

- `expiry_date` (DATE, NOT NULL)
- `total_units_produced` (INT, CHECK(total_units_produced > 0))
- `recall_status` (VARCHAR(20), DEFAULT 'ACTIVE' - CHECK in ('ACTIVE', 'UNDER_INVESTIGATION', 'RECALLED'))

4. **`hospitals`**:
   - `hospital_id` (INT, PK, AUTO_INCREMENT)
   - `hospital_name` (VARCHAR(200), NOT NULL)
   - `city` (VARCHAR(100), NOT NULL)
   - `state` (VARCHAR(100))
   - `contact_phone` (VARCHAR(20))
5. **`batch_distribution`**:
   - `distribution_id` (INT, PK, AUTO_INCREMENT)
   - `batch_number` (VARCHAR(50), FK -> `batches.batch_number`)
   - `hospital_id` (INT, FK -> `hospitals.hospital_id`)
   - `quantity_delivered` (INT, CHECK(quantity_delivered > 0))
   - `dispatch_date` (DATE, NOT NULL)

5. NoSQL Document Schema (MongoDB: `adverse_reports`)

Each adverse reaction document contains patient metadata, variable symptoms arrays, severity metrics, and optional free-text notes:

```
json
{
  "_id": "ObjectId('650f1a9b2c3d4e5f6a7b8c9d')",
  "report_id": "ADR-2026-00492",
  "batch_number": "BCH-PCM-9042",
  "medicine_id": 104,
  "medicine_name": "Cardiomax 50mg",
  "reporter_type": "HOSPITAL_DOCTOR",
  "patient_demographics": {
    "age": 58,
    "gender": "Female",
    "pre_existing_conditions": ["Hypertension", "Type 2 Diabetes"]
  },
  "symptoms": [
    { "name": "Severe Dizziness", "onset_minutes": 45, "is_critical": true },
    { "name": "Acute Hypotension", "onset_minutes": 60, "is_critical": true },
    { "name": "Nausea", "onset_minutes": 30, "is_critical": false }
  ],
  "severity_score": 5,
  "hospital_id": 12,
  "clinical_notes": "Patient experienced sudden blood pressure drop post-administration of
morning dosage. Vital stabilization required IV saline infusion.",
  "reported_at": "2026-10-06T09:30:00Z"
}
```

6. Advanced Database Queries & Operations

6.1 SQL Stored Procedure: Atomic Emergency Batch Recall

Atomically updates the batch status, logs the recall audit, and identifies affected stock distribution:

```sql
DELIMITER //
CREATE PROCEDURE sp_recall_batch(
    IN p_batch_number VARCHAR(50),
    IN p_reason VARCHAR(255),
    OUT p_affected_hospitals INT,
    OUT p_total_units_recalled INT
)
BEGIN
    -- 1. Verify and update batch status
    UPDATE batches
    SET recall_status = 'RECALLED'
    WHERE batch_number = p_batch_number;

    -- 2. Aggregate affected distribution
    SELECT
        COUNT(DISTINCT hospital_id),
        COALESCE(SUM(quantity_delivered), 0)
    INTO
        p_affected_hospitals,
        p_total_units_recalled
    FROM batch_distribution
    WHERE batch_number = p_batch_number;
END //
DELIMITER ;
```

6.2 SQL Advanced Analytical Query: High-Risk Inventory Tracing

Uses Common Table Expressions (CTE) and multi-table `JOIN` with `GROUP BY ... HAVING` to pinpoint hospitals currently holding active stock of flagged batches:

```sql
WITH FlaggedBatches AS (
    SELECT batch_number, medicine_id, expiry_date
    FROM batches
    WHERE recall_status = 'RECALLED'
)
SELECT
    h.hospital_name,
    h.city,
    m.commercial_name,
    fb.batch_number,
    SUM(bd.quantity_delivered) AS total_quarantine_units
FROM FlaggedBatches fb
JOIN batch_distribution bd ON fb.batch_number = bd.batch_number
JOIN hospitals h ON bd.hospital_id = h.hospital_id
JOIN medicines m ON fb.medicine_id = m.medicine_id
GROUP BY h.hospital_name, h.city, m.commercial_name, fb.batch_number
HAVING SUM(bd.quantity_delivered) > 0
ORDER BY total_quarantine_units DESC;
```

6.3 MongoDB Multi-Stage Aggregation Pipeline: Safety Signal Detection

Unwinds polymorphic symptom arrays to calculate symptom frequency and average severity per medicine:

```
python
pipeline = [
    # Stage 1: Match recent reports with significant severity
    {"$match": {"severity_score": {"$gte": 3}}},

    # Stage 2: Deconstruct symptoms array for granular aggregation
    {"$unwind": "$symptoms"},

    # Stage 3: Group by drug and symptom to compute frequencies
    {
        "$group": {
            "_id": {
                "medicine_name": "$medicine_name",
                "symptom": "$symptoms.name"
            },
            "incident_count": {"$sum": 1},
            "avg_severity": {"$avg": "$severity_score"},
            "critical_cases": {
                "$sum": {"$cond": ["$symptoms.is_critical", 1, 0]}
            }
        }
    },

    # Stage 4: Filter out low-frequency noise (Signal Detection threshold)
    {"$match": {"incident_count": {"$gte": 2}}},

    # Stage 5: Format output projection
    {
        "$project": {
            "_id": 0,
            "medicine": "$_id.medicine_name",
            "symptom": "$_id.symptom",
            "incident_count": 1,
            "avg_severity": {"$round": ["$avg_severity", 2]},
            "critical_cases": 1,
            "signal_risk_score": {
                "$multiply": ["$incident_count", "$avg_severity"]
            }
        }
    },

    # Stage 6: Rank by calculated safety risk
    {"$sort": {"signal_risk_score": -1}}
]
```

6.4 MongoDB Text Search Index

Enables clinical keyword retrieval across doctor notes without needing external NLP libraries:

```
python
# Setup text index in MongoDB
db.adverse_reports.create_index([("clinical_notes", "text")])

# Fast keyword retrieval
results = db.adverse_reports.find(
    {"$text": {"$search": "anaphylaxis seizure arrhythmia"}},
    {"score": {"$meta": "textScore"}}
).sort([("score", {"$meta": "textScore"})])
```

7. Python Implementation Architecture (OOP & File I/O)

7.1 Object-Oriented Domain Models (`app/models/`)

- **`Medicine`** & **`Batch`**: Encapsulates medicine attributes, expiry calculations (`is_expired()`), and recall state machine.
- **`AdverseReport` (Pydantic Model)**: Enforces input validation (e.g., `severity_score` constrained to $[1, 5]$[1, 5], age constrained to $[0, 120]$[0, 120], required batch format regex).
- **`SafetyAuditor`**: Encapsulates safety evaluation logic and triggers automatic threshold alerts.

7.2 Custom Domain Exceptions (`app/utils/exceptions.py`)

- `ExpiredBatchDispenseError`: Raised if distribution is attempted for an expired batch.
- `RecalledBatchOperationError`: Prevents logging new distribution for batches already under recall.
- `DatabaseConnectionError`: Wraps database driver connection outages.
- `InvalidSeverityScoreError`: Raised when severity input is outside valid boundaries.

7.3 File Handling & Automated Reporting (`app/services/report_generator.py`)

- **Input CSV Bulk Ingestion**: Pharmacists can upload a batch shipment `.csv` file via Streamlit; the system parses, validates schemas, and inserts records into SQL within a single transaction.
- **Output PDF / CSV Export**: Safety officers can generate a downloadable **Safety Audit Recall Manifest** containing quarantined quantities, hospital contacts, and adverse event summaries using Python's `csv` module and `reportlab` or `fpdf2`.

8. Multi-Portal User Interface (Streamlit)

```
Pharmavigil Dashboard
├── 1. Doctor & Patient Portal
│   ├── Structured Medicine & Batch Dropdowns
│   ├── Multi-select Symptoms & Severity Slider (1–5)
│   ├── Optional Doctor Clinical Notes Text Area
│   └── Instant Confirmation Receipt
│
├── 2. Hospital Pharmacist Inventory Portal
│   ├── Live Batch Stock Table (SQL JOIN query)
│   ├── CSV Bulk Batch Intake Uploader
│   ├── Expiration Date Alarms & Warnings
│   └── Dispatch to Regional Departments
│
└── 3. Safety Officer & Admin Control Center
    ├── Live Real-Time Risk Alarms (Signal Detection Pipeline)
    ├── Aggregation Charts (Top Adverse Symptoms by Drug)
    ├── Emergency Batch Recall Button (Calls Stored Procedure)
    ├── Clinical Keyword Search Box (MongoDB Text Index)
    └── One-Click Safety Audit Manifest Download (.pdf / .csv)
```

9. Recommended Directory Structure

```
text
DCDS Project/
│
├── app/                                # Application package
│   ├── __init__.py
│   ├── config.py                       # DB connection URI & environment variables
│   │
│   ├── database/                       # Database driver layer
│   │   ├── __init__.py
│   │   ├── sql_db.py                   # MySQL / PostgreSQL connection helper
│   │   └── mongo_db.py                 # PyMongo connection helper
│   │
│   ├── models/                         # OOP classes & Pydantic schemas
│   │   ├── __init__.py
│   │   ├── medicine.py                 # Medicine & Batch OOP domain classes
│   │   └── schemas.py                  # Pydantic schemas for input validation
│   │
│   ├── services/                       # Business logic layer
│   │   ├── __init__.py
│   │   ├── sql_service.py              # SQL queries, Joins & Stored Procedure calls
│   │   ├── mongo_service.py            # MongoDB aggregation pipelines & text search
│   │   └── report_generator.py         # File Handling: CSV import & PDF/CSV export
│   │
│   ├── utils/                          # Utilities & custom exceptions
│   │   ├── __init__.py
│   │   └── exceptions.py               # Custom domain exceptions
│   │
│   └── ui/                             # Streamlit frontend views
│       ├── app.py                      # Main Streamlit navigation entrypoint
│       ├── page_patient.py             # Adverse reaction reporting form
│       ├── page_inventory.py           # Hospital batch inventory management
│       └── page_admin.py               # Safety officer alarms & recall center
│
├── sql_scripts/                        # SQL initialization and DDL
│   ├── schema.sql                      # DDL: CREATE TABLE, Constraints, Keys
│   ├── stored_procedures.sql           # CREATE PROCEDURE sp_recall_batch
│   └── seed_data.sql                   # Realistic demo data for presentation
│
├── data/                               # File handling demo files
│   ├── sample_batches.csv              # Input CSV for batch intake demonstration
│   └── exports/                        # Destination for generated audit reports
│
├── tests/                              # Automated testing suite (Pytest)
```

```
│   ├── __init__.py
│   ├── test_validation.py              # Tests for Pydantic input models
│   ├── test_sql_operations.py          # Tests for SQL queries & stored procedures
│   └── test_mongo_pipeline.py          # Tests for MongoDB aggregations
│
├── requirements.txt                    # Project dependencies
├── .env.example                        # Sample configuration file
├── PROJECT_PLAN.md                     # High-level syllabus & rubric alignment
├── PHARMAVIGIL_PROJECT_GUIDE.md        # This comprehensive engineering specification
└── README.md                           # Quick start guide & execution commands
```

10. Viva & Evaluation FAQ (How to Defend Your Project)

1. **Q: Why did you use both SQL and NoSQL instead of putting everything in SQL?**
    - **A:** *"Pharmaceutical inventory and distribution must adhere to strict regulatory compliance, relational integrity, and ACID guarantees (foreign keys, non-null expiration dates, non-negative stock counts). However, patient adverse event telemetry is inherently polymorphic and semi-structured—symptom counts vary, severity metrics differ, and clinical observations are unstandardized. Using MongoDB for telemetry avoids continuous schema migrations in SQL while preserving transactional guarantees in PostgreSQL/MySQL for physical batch inventory."*
2. **Q: How did you implement advanced database concepts in SQL?**
    - **A:** *"We implemented atomic stored procedures (`sp_recall_batch`) to transition batch statuses and calculate affected distribution within an isolated transaction. Additionally, we utilized Common Table Expressions (CTEs) and multi-table `JOIN`s with `GROUP BY ... HAVING` to locate quarantined units across distributed hospital networks."*
3. **Q: How did you implement advanced database concepts in MongoDB?**
    - **A:** *"We built a multi-stage aggregation pipeline utilizing `$match`, `$unwind`, `$group`, `$project`, and `$sort` to execute pharmacovigilance signal detection across nested symptom documents. Furthermore, we built a native MongoDB Text Index across doctor clinical notes for keyword retrieval."*
4. **Q: How did you handle doctor clinical notes without complex AI?**
    - **A:** *"Clinical notes are stored in MongoDB document strings. While standard symptom reporting uses structured inputs for predictable querying, doctor free-text notes are indexed using MongoDB's native full-text search engine (`$text` index), allowing instant clinical keyword search without unnecessary NLP overhead."*

5. **Q: How did you ensure security and validation?**
    - **A:** *"All SQL interactions utilize parameterized queries via Python DB-API to eliminate SQL injection risks. All incoming adverse reports are validated at the application boundary via Pydantic schemas, checking bounded ranges on severity scores (*1 to 5*1 to 5) and patient age (*0 to 120*0 to 120)."*